Agentic AI Systems for Insider Trading Sentiment Analysis: Architectural Designs and Technical Evaluations

Executive Summary

The paradigm of artificial intelligence within quantitative finance is undergoing a structural transformation, migrating from generative models that produce static textual responses to agentic systems that autonomously orchestrate multi-step, complex workflows. Agentic artificial intelligence systems possess the capacity to perceive environments, formulate strategic plans, execute sequential actions through the use of external tools, and iteratively adapt their behavior based on continuous environmental feedback. In highly dynamic domains such as financial market analysis, this autonomy facilitates the real-time synthesis of highly disparate, high-velocity data streams into actionable market intelligence.

The primary objective of this report is to architect an advanced, multi-agent backend system capable of conducting autonomous insider trading analysis and correlating these findings with public sentiment. Specifically, the system is engineered to ingest United States Securities and Exchange Commission (SEC) Form 4 filings representing the trailing twenty-four hours of market activity, execute arithmetic logic to isolate the top five equity transactions by absolute dollar volume, and subsequently extract seven days of corresponding social sentiment data from the X (formerly Twitter) platform. This aggregated intelligence must be stored within an advanced, highly structured knowledge repository and exposed to end-users via a zero-hallucination, context-aware chatbot interface capable of generating dynamic visualizations on demand. Furthermore, the architecture must implement a closed learning loop utilizing the Hermes Agent Framework to ensure the system autonomously optimizes its procedural execution over time.

This comprehensive analysis systematically deconstructs the problem space into discrete operational layers, evaluates the successes and failures of existing real-world financial AI implementations, and conducts a rigorous comparative analysis of the underlying technology stack. Special emphasis is placed on evaluating knowledge representation frameworks—specifically comparing traditional Retrieval-Augmented Generation (RAG) against vectorless tree-search methodologies like PageIndex and graph-enhanced retrieval like LightRAG. The report ultimately proposes three distinct architectural designs: a hierarchical modular pipeline, an asynchronous event-driven swarm, and a hybrid neuro-symbolic system. Through a detailed analysis of data flow, scalability constraints, and risk mitigation strategies, this document provides a definitive recommendation for deploying a robust, production-grade financial intelligence system.

Problem Decomposition

The construction of an autonomous financial intelligence system necessitates the decomposition of the overarching objective into highly specialized, interoperable layers. Monolithic pipelines are inherently fragile in agentic contexts; a failure in data parsing should not induce a catastrophic failure in the conversational interface. By compartmentalizing the system into five foundational layers—Data, Processing, Intelligence, Interface, and Learning—the architecture achieves the modularity required for resilient, autonomous operation.

The Data Layer

The Data Layer represents the sensory apparatus of the agentic system, responsible for the continuous, high-fidelity ingestion of external information across two primary vectors: regulatory disclosures and social discourse.

The regulatory vector focuses exclusively on SEC Form 4 filings. Under Section 16 of the Securities Exchange Act of 1934, senior executives, corporate directors, and shareholders possessing more than a ten percent equity stake in a publicly traded company are legally mandated to report any changes in their beneficial ownership.[1] These disclosures must be submitted to the SEC's Electronic Data Gathering, Analysis, and Retrieval (EDGAR) database within two business days of the transaction. The Data Layer must interface with the EDGAR ecosystem to isolate all Form 4 filings published within a rolling twenty-four-hour window. This extraction process is non-trivial; Form 4 filings contain complex Extensible Business Reporting Language (XBRL) and XML structures detailing both derivative and non-derivative transactions.[2] The system must accurately ingest these raw structures, capturing variables such as the transaction date, the security title, the volume of shares acquired or disposed of, and the execution price.

The social vector is dedicated to extracting real-time public sentiment from the X (Twitter) platform. For each of the top five tickers identified during the regulatory processing phase, the Data Layer must execute scraping protocols to retrieve historical posts encompassing the preceding seven days. Social media extraction presents unique challenges, including navigating dynamic web architectures, managing API rate limits, and circumventing automated bot detection mechanisms.[3] The system leverages Apify's serverless infrastructure to aggregate a statistically significant sample of cashtag mentions, reply threads, and associated engagement metrics, creating a raw corpus of unstructured public discourse.

The Processing Layer

Raw intelligence possesses minimal utility without rigorous standardization and enrichment. The Processing Layer serves as the analytical engine, transforming unstructured text and complex XML into structured, quantifiable datasets.

For the regulatory data stream, the Processing Layer executes the arithmetic logic required to rank the insider trades. A single Form 4 filing frequently contains multiple rows denoting

separate transactions. The processing algorithms must calculate the absolute dollar value of each transaction by multiplying the share volume by the execution price, subsequently summing the disposal values and subtracting the acquisition values to determine the net capital flow.[2] Furthermore, the system must account for structural redundancies within the EDGAR database. It is common for both the reporting insider (e.g., the Chief Executive Officer) and the issuing corporation to submit parallel disclosures for the same transaction, creating "double-ups" that must be deduplicated to ensure accurate volumetric ranking.[2]

Concurrently, the social media data stream undergoes natural language processing to assign sentiment polarity scores. The Processing Layer evaluates the scraped tweets, classifying the localized sentiment as positive, negative, or neutral. This phase requires sophisticated contextual awareness to differentiate between standard vernacular and financial jargon, ensuring that a "bearish" technical analysis is correctly identified as negative sentiment regarding the asset's price action.[5] The output of this layer is a synchronized, structured dataset linking definitive insider capital flows with quantified public perception.

The Intelligence Layer

The Intelligence Layer functions as the cognitive repository of the agentic system, governing knowledge representation, semantic retrieval, and the coordination of multi-agent workflows. A strict operational constraint for financial artificial intelligence is the complete elimination of algorithmic hallucinations; therefore, the Intelligence Layer must ensure that all generative reasoning is strictly grounded in the ingested dataset.

This layer dictates how the processed SEC filings and Twitter data are indexed for long-term recall. Traditional retrieval systems rely on chunking text into arbitrary token limits and mapping them to dense vector embeddings. However, the Intelligence Layer of the proposed system must evaluate advanced frameworks that preserve the structural integrity of financial tables and the relational complexity of social networks. Advanced methodologies such as PageIndex—which constructs a hierarchical semantic tree of the document—and LightRAG—which builds a comprehensive knowledge graph of entities and themes—are evaluated to guarantee high-precision, multi-hop reasoning capabilities.[6] The Intelligence Layer also manages the routing protocols, determining which specialized agent is best suited to query these disparate knowledge stores based on the user's prompt.

The Interface Layer

The Interface Layer translates the complex, asynchronous operations of the backend into an intuitive, accessible format for the end-user. It features a conversational chatbot engine that interacts dynamically with the Intelligence Layer to construct verifiable responses. The conversational agent is explicitly instructed to bypass its internal parametric memory, relying solely on the retrieved context to formulate answers, thereby maintaining a zero-hallucination environment.

In addition to textual synthesis, the Interface Layer encompasses an autonomous visualization

module. When a user requests a graphical representation of the data—such as a time-series overlay of an insider's transaction history plotted against a seven-day social sentiment moving average—the interface layer orchestrates sandboxed code-execution tools. The agent dynamically generates and executes Python-based charting logic using libraries such as Plotly or Matplotlib, returning rendered visual artifacts directly into the chat interface.[8]

The Learning Layer

The Learning Layer elevates the architecture from a static procedural software pipeline to a genuinely autonomous, evolving entity. Utilizing the Hermes Agent Framework, the system implements a continuous, closed-loop self-improvement mechanism.[10] As the multi-agent system navigates novel edge cases—such as parsing an anomalous Form 4 footnote structure or handling complex rate-limiting behaviors from the social scraper—the Learning Layer evaluates the execution trajectory. Upon successful completion of a complex task, the system autonomously abstracts the successful reasoning path into a persistent, reusable procedural memory asset, formatted as a markdown skill file containing explicit instructions and shell commands.[11] This programmatic reflection allows the system to iteratively optimize its internal pathways, reducing latency and reliance on raw generative reasoning in subsequent iterations.

Exploration of Existing Real-World Systems

To architect a robust solution, it is imperative to analyze the deployment patterns, successes, and inherent failures of existing real-world agentic financial systems. The transition from predictive machine learning to autonomous decision-making in finance has highlighted several critical architectural lessons.

Enterprise financial institutions are increasingly deploying agentic artificial intelligence to manage credit underwriting, optimize portfolios, and conduct forensic analysis. Implementations such as Moody's CreditView leverage agentic systems to continuously monitor financial statements, sentiment signals, and rating changes in real-time, autonomously triggering pre-approved risk adjustments when systemic anomalies are detected.[13] Similarly, retail-focused platforms like Stonki.ai utilize multi-agent swarms to analyze market data, cross-referencing social mentions against technical setups and generating probabilistic trading plans.[15] A consistent architectural pattern across these successful deployments is the strict enforcement of agent separation. Developers of production systems, such as Replicats.ai, note that assigning distinct, non-overlapping roles for data generation, risk management, and execution prevents "agent confusion," a phenomenon where a single large language model attempts to balance conflicting objectives, resulting in degraded performance.[16]

However, the deployment of Retrieval-Augmented Generation (RAG) in financial contexts has exposed severe structural limitations. Traditional RAG relies on semantic similarity, which frequently fails when querying dense financial documents. The arbitrary chunking of texts destroys the inherent relationships within financial tables, separating headers from their corresponding numerical cells. This "chunking war" results in traditional vector-based RAG

systems scoring a mere thirty to fifty percent accuracy on rigorous financial evaluation datasets like FinanceBench.[17] In contrast, experimental systems utilizing reasoning-based retrieval, such as VectifyAI's Mafin 2.5, have achieved up to 98.7 percent accuracy by replacing vector databases with hierarchical tree structures, proving that structural preservation is paramount for financial query accuracy.[19]

Furthermore, the implementation of autonomous learning loops presents unique operational risks. While the theoretical promise of a self-improving system is immense, real-world evaluations of the Hermes Agent Framework have identified the "overconfident evaluator problem." In closed learning loops where the agent is responsible for judging its own performance before committing a new skill to memory, the agent frequently exhibits a bias toward positive self-evaluation.[21] If the agent hallucinates data during an extraction task, it may erroneously classify the hallucinated trajectory as a success, permanently extracting a flawed procedural skill that corrupts future executions. This failure mode necessitates the introduction of deterministic validation gates—relying on symbolic logic rather than neural generation—to verify outcomes before they are integrated into the agent's long-term memory.[22]

Technology Evaluation

The efficacy of the proposed system is contingent upon the meticulous selection of technologies for knowledge representation, data extraction, and multi-agent orchestration. The following analysis evaluates the primary candidates against the system's operational constraints.

Knowledge Representation and Retrieval

The requirement for absolute precision and the elimination of generative hallucinations necessitates a departure from naive retrieval pipelines. The system must navigate the trade-offs between traditional vector indexing, semantic tree navigation, and knowledge graph extraction.

Retrieval Methodology	Core Mechanism	Primary Advantages	Limitations	Optimal Use Case
Traditional RAG	Token-based chunking and vector embedding. Uses cosine similarity to retrieve text fragments.[18]	High scalability, sub-second latency, low computational cost.[23]	Destroys structural context; fails on tables and cross-references; poor multi-hop reasoning.[20]	Broad semantic search across massive, unstructured text corpora.
PageIndex	Vectorless	Exceptional	Extremely high	Precise

	retrieval. Constructs a hierarchical JSON tree of document sections. LLM explicitly navigates the tree.[6]	accuracy (98.7% on FinanceBench) ; preserves table structures; explainable retrieval paths.[18]	latency (multiple sequential LLM calls); does not scale well to thousands of discrete documents.[25]	extraction from dense, highly structured individual documents (e.g., SEC Form 4).
LightRAG	Graph-enhanced retrieval. Extracts entities and relationships to build a knowledge graph combined with vector embeddings.[27]	Excellent at abstract, multi-hop thematic queries; supports efficient incremental updates without rebuilding the graph.[7]	Higher initial processing cost during ingestion compared to naive RAG; complex implementation .[27]	Mapping relational, dynamic, and unstructured data streams (e.g., Twitter social sentiment).

Given the bipartite nature of the data ingestion, a hybrid retrieval architecture is optimal. PageIndex is definitively the superior choice for indexing the highly structured SEC Form 4 filings. The JSON tree structure ensures that the relationships between reporting owners, transaction codes, and absolute values remain intact, preventing the exact numerical extraction errors that plague vector systems. Conversely, LightRAG provides the optimal mechanism for the social media data. Twitter sentiment is inherently relational and chaotic; understanding the connections between specific executives, cashtags, and broader market events requires graph traversal. LightRAG's ability to efficiently ingest incremental updates makes it perfectly suited for processing rolling seven-day sentiment feeds without necessitating computationally ruinous full-graph rebuilds.

Data Pipeline Design

The extraction of regulatory and social data requires programmatic interfaces that seamlessly integrate with agentic workflows.

For the regulatory data, the system evaluates several robust open-source python libraries. The sec-api-python package provides a highly stable Query API utilizing Lucene syntax to fetch real-time metadata and XML documents.[29] While effective, integrating this raw API requires the development of extensive custom python wrapper tools for the language model. Alternatively, the sec-edgar-agentkit provides an inherently AI-native architecture. Built directly upon the

sec-edgar-mcp (Model Context Protocol) server, it exposes pre-configured natural language tools optimized for frameworks like LangChain and Hermes.[31] By leveraging the MCP standard, the agent can autonomously format queries, look up Central Index Keys (CIKs), and parse insider trading transactions with minimal integration overhead. Tools such as the insider-trading-analyzer offer excellent historical backtesting and local DuckDB storage, but lack the real-time, conversational MCP bindings necessary for dynamic chatbot orchestration.[32] Therefore, sec-edgar-agentkit is selected as the primary regulatory data conduit.

For social sentiment extraction, the Apify platform offers unparalleled scalability through its serverless "Actor" ecosystem. The twitter-x-tweets-scraper Actor captures exhaustive metadata, including timestamps, engagement metrics, and complete textual content for specified search queries.[3] Crucially, Apify provides native MCP server integration. This allows the AI agent to independently configure scraping parameters, execute the web scraping tasks, and retrieve the resulting datasets without manual prompting.[3]

Agent Orchestration Strategy

The orchestration of specialized sub-agents requires a framework capable of managing state transfers, tool execution, and long-term memory. The Hermes Agent Framework, developed by Nous Research, offers highly specialized capabilities engineered specifically for autonomous, long-horizon operations.[10]

Framework Feature	Technical Implementation	Architectural Benefit
Model Context Protocol (MCP)	Standardized bridge for connecting external tool servers.[34]	Allows seamless integration with both sec-edgar-agentkit and Apify without custom Python wrappers.
Cross-Session Recall	SQLite FTS5 combined with LLM summarization.[10]	Enables the agent to maintain contextual awareness of past financial queries and user preferences.
Programmatic Tool Calling	execute_code functionality collapses multi-step pipelines into single inference calls.[10]	Empowers the agent to dynamically generate and execute Python code (e.g., Matplotlib/Plotly) in a secure sandbox.

Closed Learning Loop	Autonomous evaluation and extraction of complex trajectories into reusable SKILL.md files.[10]	Transforms the system from a static pipeline into an evolving entity that reduces latency on repeated tasks.

The Hermes framework's native learning loop is the deciding factor. By allowing the agent to continuously refine its approach to handling SEC data anomalies or scraping timeouts, the system inherently scales its efficiency over time.

Architectural Designs

To fulfill the requirements for a modular, scalable, and multi-agent financial intelligence system, three distinct architectural paradigms are proposed. Each approach offers specific advantages regarding latency, deterministic accuracy, and implementation complexity.

Architecture 1: Modular Agent Pipeline (Hierarchical Orchestration)

The Modular Agent Pipeline represents a classic hierarchical orchestration model. A centralized "supervisor" agent acts as the primary routing node, interpreting user intent and sequentially delegating operations to highly specialized subordinate agents. This architecture prioritizes deterministic execution and clear operational boundaries.[36]

Architecture Diagram (Textual)

1. **User Interface** -> Transmits query to the **Supervisor Agent**.
2. **Supervisor Agent** -> Delegates task to the **Regulatory Analysis Agent**.
3. **Regulatory Analysis Agent** -> Interfaces via MCP with sec-edgar-agentkit to fetch the trailing 24 hours of Form 4 filings. It executes arithmetic tools to calculate and return the top 5 tickers by absolute dollar value.
4. **Supervisor Agent** -> Receives tickers and delegates to the **Social Intelligence Agent**.
5. **Social Intelligence Agent** -> Invokes the Apify MCP Server, triggering the X (Twitter) Scraper Actor for the specified tickers (7-day lookback).
6. **Social Intelligence Agent** -> Executes natural language processing to assign polarity scores to the scraped text, returning the enriched dataset.
7. **Supervisor Agent** -> Delegates to the **Knowledge Compiler Agent**, which builds a hybrid PageIndex and LightRAG knowledge store.
8. **Supervisor Agent** -> Delegates to the **Conversational Agent** for zero-hallucination query resolution, or the **Visualization Agent** for dynamic Matplotlib/Plotly chart generation via sandboxed Python execution.
9. **Hermes Background Loop** -> The framework monitors the sequential trajectory, subsequently updating procedural memory (SKILL.md) for future optimizations.

Agent Roles and Responsibilities

Agent Designation	Core Responsibility	Input	Output
Supervisor Agent	Task decomposition and sequential routing.[36]	User natural language prompt.	Delegated sub-tasks and finalized synthesis.
Regulatory Analysis Agent	MCP interaction with SEC EDGAR; parsing XML/XBRL.	Date range constraints.	JSON array of Top 5 ranked tickers and values.
Social Intelligence Agent	Managing Apify Actor execution and sentiment classification.	Ticker symbols.	Enriched dataset of sentiment-scored tweets.
Knowledge Compiler Agent	Formats data into semantic trees and graph databases.	Raw regulatory and social datasets.	Indexed PageIndex / LightRAG directories.
Visualization Agent	Writes and executes Python code to render charts.[8]	Aggregated data arrays.	Base64 encoded images or HTML graphics.

Analysis

This architecture offers unparalleled control and determinism. The linear, sequential flow guarantees that the Social Intelligence Agent does not expend API credits scraping Twitter data until the Regulatory Analysis Agent has definitively verified the top five SEC tickers. This strict separation of concerns minimizes "agent confusion" and simplifies debugging, as failures can be isolated to a specific sub-agent.[16] However, the reliance on a central supervisor creates a single point of computational bottleneck.[36] The system cannot parallelize the scraping operations, resulting in significant latency spikes during the data ingestion phase. While scalable in complexity, its throughput speed is inherently limited by the sequential nature of the workflow.

Architecture 2: Event-Driven Asynchronous Swarm

The Event-Driven Swarm architecture discards the central supervisor in favor of a decentralized, asynchronous event bus. Agents operate as independent microservices, subscribing to specific data events, executing their specialized tasks in parallel, and publishing their results back to the broader system. This design is inspired by the low-latency infrastructure utilized in

high-frequency algorithmic trading.[16]

Architecture Diagram (Textual)

1. **Background Cron Scheduler** -> Periodically triggers the **SEC Ingestion Agent**.
2. **SEC Ingestion Agent** -> Continuously monitors the EDGAR feed. Upon detecting a high-value insider trade, it publishes a global message: ``.
3. **Event Bus** -> Broadcasts the event to all active subscribers.
4. **Social Swarm Workers** (Multiple Instances) -> Subscribe to the event. Upon receipt, a worker node immediately triggers an Apify scraping container for the specific ticker, performs sentiment analysis, and publishes: ``.
5. **Continuous Compilation Agent** -> Listens for completion events, asynchronously updating the LightRAG knowledge graph and reconstructing the PageIndex trees in the background.
6. **Interface Chat Agent** -> Operates entirely independently of the ingestion pipeline. It responds to user queries by searching the pre-compiled, real-time knowledge stores.
7. **Visualization Agent** -> Subscribes to `` triggered by the Chat Agent, generating the required visual artifacts on demand.

Agent Roles and Responsibilities

Agent Designation	Core Responsibility	Input	Output
SEC Ingestion Agent	Continuous polling of regulatory APIs.	Polling intervals.	Form 4 event publications.
Social Swarm Workers	Highly parallelized execution of Apify web scrapers.	Ticker event payloads.	Sentiment event publications.
Continuous Compilation Agent	Background maintenance of vector, tree, and graph databases.	Aggregated event data.	Updated knowledge storage indices.
Interface Chat	Real-time user interaction utilizing	User prompts.	Zero-latency grounded

Agent	pre-compiled knowledge.		responses.

Analysis

The primary advantage of the swarm architecture is exceptional speed and horizontal scalability. By entirely decoupling the data ingestion pipeline from the user interface, conversational queries experience near-zero latency, as the knowledge base is compiled and updated asynchronously in the background. The system can dynamically spin up dozens of Social Swarm Workers to process high volumes of filings during earnings seasons.[36] However, this architecture introduces immense systemic complexity. Managing state across a decentralized event bus requires robust conflict resolution protocols to prevent race conditions—situations where multiple agents attempt to write data to the LightRAG graph simultaneously.[38] Furthermore, the lack of centralized oversight makes debugging complex multi-agent failures significantly more challenging.

Architecture 3: Hybrid Neuro-Symbolic Memory-Enhanced System

The Hybrid Neuro-Symbolic architecture synthesizes the capabilities of generative neural networks (LLMs) with the strict determinism of classical symbolic logic (Python execution). It places the Hermes closed learning loop at the absolute center of the operation, utilizing deterministic validation gates to eliminate arithmetic hallucinations before data is committed to the dual-tiered memory stores.[22]

Architecture Diagram (Textual)

1. **User Request** -> Intercepted by the **Hermes Orchestrator Agent**.
2. **Hermes Orchestrator Agent** -> Queries its internal **FTS5 Procedural Memory** to locate a previously validated SKILL.md containing the optimal code path for SEC and Apify data extraction.[10]
3. **Exploration vs. Exploitation** -> If no skill exists, the Orchestrator spawns a sub-agent to dynamically generate the MCP calls. If a skill exists, the Orchestrator executes the stored procedure to retrieve the raw data.[12]
4. **Symbolic Validation Gate** -> The raw data is routed to a deterministic Python sandbox. This step bypasses the LLM entirely; standard Pandas libraries are used to mathematically calculate net insider volume, rank the top 5 tickers, and deduplicate entities.[40]
5. **Memory Construction Agent** -> Receives mathematically verified data. It routes the structured SEC filings to **PageIndex** for hierarchical tree construction, and the relational Twitter sentiment data to **LightRAG** for graph indexing.
6. **Reasoning Agent** -> Processes the initial user request using the dual-tiered memory. It leverages LightRAG to synthesize broad market sentiment connections and utilizes

PageIndex to extract exact numerical citations.[20]

7. **Visualization Sub-Routine** -> The Reasoning Agent executes a verified code skill to render the Matplotlib charts using the symbolically validated data arrays.
8. **Learning Loop Update** -> The framework evaluates the success of the trajectory. Because the intermediate data was symbolically validated, the framework can safely extract the successful reasoning path into a permanent memory skill without the risk of reinforcing hallucinated logic.[12]

Agent Roles and Responsibilities

Agent Designation	Core Responsibility	Input	Output
Hermes Orchestrator Agent	Managing exploration, procedural memory recall, and task dispatch.[10]	User requests.	Strategic execution plans.
Symbolic Validation Gate	Deterministic mathematical ranking and verification (Non-LLM).[40]	Raw, unranked JSON data.	Mathematically validated DataFrames.
Memory Construction Agent	Routing data to optimal indexing structures (PageIndex vs. LightRAG).	Validated datasets.	Highly structured retrieval endpoints.
Reasoning Agent	Synthesizing intelligence across graphs and hierarchical trees.	System indices and user prompts.	Grounded, cited financial intelligence.

Analysis

This architecture provides the highest degree of reliability and accuracy for high-stakes financial environments. By forcing raw data through a deterministic Symbolic Validation Gate, the system entirely neutralizes the most common failure mode of financial AI: LLM arithmetic hallucination.[37] The dual-retrieval mechanism ensures that the system utilizes the optimal

indexing strategy for each specific data type, maximizing precision. While managing dual memory architectures (tree and graph) introduces significant computational overhead, the continuous integration of the Hermes learning loop mitigates this over time. As the system proceduralizes complex tasks into static Python code skills, its reliance on expensive generative reasoning decreases, creating a system that becomes exponentially faster and cheaper as it matures.[10]

Optimization & Risks

The deployment of autonomous financial intelligence systems introduces specific systemic vulnerabilities that require proactive mitigation strategies.

Bottlenecks and Latency Management

The most significant computational bottleneck within the proposed architecture lies in the PageIndex retrieval strategy. Because PageIndex abandons instantaneous vector math in favor of iterative LLM navigation through a JSON tree, deep queries over complex SEC documents can induce latencies exceeding twenty-five seconds.[20] To mitigate this degradation in user experience, the system must implement intelligent routing heuristics at the Interface Layer. Queries requiring exact, granular numerical extraction (e.g., "What was the exact volume of non-derivative shares sold?") should be routed to the slower PageIndex. Conversely, broad thematic inquiries (e.g., "Summarize the overall social perception of the CEO's recent trades") should be routed exclusively to the highly efficient LightRAG graph network.[18] Additionally, the Apify extraction phase introduces unavoidable delays due to API rate limits and network latency. The orchestrator must execute Apify Actors asynchronously, utilizing webhook callbacks to alert the system upon completion rather than blocking the primary execution thread.[3]

Data Leakage and Hallucination Risks

In quantitative finance, the phenomenon of data leakage—where a language model inadvertently introduces external, unverified knowledge from its parametric pre-training that contradicts the real-time dataset—can precipitate catastrophic analytical errors. Furthermore, foundational models exhibit systemic fragility when tasked with arithmetic operations across large datasets. To neutralize these risks, the architecture mandates the strict utilization of "tool-call gatekeepers".[40] The language model is explicitly barred from performing internal calculations. All mathematical logic, such as netting the absolute value of Form 4 disposals, must be delegated to a deterministic Python execution sandbox via the Hermes framework's native execute_code capability.[10] The agent generates the data transformation logic, executes it within a secure, isolated container, and utilizes the deterministic output as the absolute source of truth.

Monitoring and Governance Strategies

Sustaining the efficacy of the Hermes closed learning loop requires comprehensive systemic observability. If an agent consistently fails to properly evaluate its own success, it risks

permanently extracting flawed procedural skills that corrupt future executions.[21] The architecture must implement a continuous Governance Telemetry Schema (GTS), integrating with frameworks like OpenTelemetry to capture the granular metadata of every agent trajectory.[43] This includes logging every MCP tool invocation, Apify extraction parameter, and PageIndex tree traversal. By establishing deterministic output checks—such as ensuring that the final output array mathematically matches the source JSON—the system can automatically quarantine anomalous runs for human-in-the-loop review. Once a developer rectifies the specific failure point, the Hermes offline reinforcement learning mechanisms can seamlessly update the agent's procedural memory base, restoring operational integrity.[12]

Final Recommendation

To satisfy the stringent requirements for a production-grade, highly autonomous financial intelligence system, **Architecture 3: Hybrid Neuro-Symbolic Memory-Enhanced System** is decisively recommended.

While the Modular Pipeline (Architecture 1) offers a straightforward implementation trajectory suitable for conceptual demonstrations, it fails to fully exploit the advanced capabilities of the requested technology stack and is hindered by sequential latency. The Event-Driven Swarm (Architecture 2) provides unparalleled parallel execution speeds, but its decentralized nature introduces substantial architectural complexities, including the management of race conditions and complex state orchestration across distributed nodes, which may destabilize the retrieval mechanisms.

Architecture 3 represents the optimal synthesis of modern artificial intelligence and classical software engineering. It directly addresses the most critical vulnerability inherent in generative AI for finance by instituting a deterministic, symbolic validation gate that algorithmically prevents LLM-induced mathematical hallucinations.[37] Furthermore, it embraces the operational realities of disparate data formats. By deploying a dual-tiered memory infrastructure, the system leverages the structural preservation of PageIndex to navigate the dense tabular data of SEC Form 4 filings [17], while simultaneously utilizing LightRAG's incremental graph architecture to dynamically map the chaotic, relational entities present in Twitter sentiment data.[27]

Most importantly, Architecture 3 seamlessly integrates the Hermes Agent Framework's core value proposition: the closed learning loop. By allowing the primary orchestrator to autonomously invoke the sec-edgar-agentkit and Apify MCP servers, and subsequently execute dynamic chart generation within sandboxed environments, the system abstracts successful operations into persistent procedural memory.[10] This architecture ensures that the backend does not remain a static software pipeline, but functions as a genuinely agentic entity capable of continuous evolution, optimization, and self-improvement in the pursuit of precise financial intelligence.

Works cited

1. Insider Transactions Data Sets - SEC.gov, accessed April 9, 2026, https://www.sec.gov/data-research/sec-markets-data/insider-transactions-data-sets
2. Insider trading visualised with Python & sec-api.io | by Dr J - Medium, accessed April 9, 2026, https://medium.com/@jan_5421/insider-trading-visualised-with-python-sec-api-io-5f12a7799b3e
3. Twitter/X Scraper — Profiles, Tweets, Followers & Following - Apify, accessed April 9, 2026, https://apify.com/automation-lab/twitter-scraper
4. Twitter (X) Scraper - Apify, accessed April 9, 2026, https://apify.com/scrapers/twitter
5. Sentiment analysis - Apify, accessed April 9, 2026, https://apify.com/use-cases/sentiment-analysis
6. VectifyAI/PageIndex - Vectorless, Reasoning-based RAG - GitHub, accessed April 9, 2026, https://github.com/VectifyAI/PageIndex
7. LightRAG, accessed April 9, 2026, https://lightrag.github.io/
8. Agent to generate Charts based on the user's prompt : r/LangChain - Reddit, accessed April 9, 2026, https://www.reddit.com/r/LangChain/comments/1cp9gqj/agent_to_generate_charts_based_on_the_users_prompt/
9. Plotly vs matplotlib: A quick comparison with visual guides | Fabi.ai, accessed April 9, 2026, https://www.fabi.ai/blog/plotly-vs-matplotlib-a-quick-comparison-with-visual-guides
10. Hermes Agent Documentation, accessed April 9, 2026, https://hermes-agent.nousresearch.com/docs/
11. hermes-agent-docs/skills.md at main - GitHub, accessed April 9, 2026, https://github.com/mudrii/hermes-agent-docs/blob/main/skills.md
12. New framework lets AI agents rewrite their own skills without retraining the underlying model, accessed April 9, 2026, https://venturebeat.com/orchestration/new-framework-lets-ai-agents-rewrite-their-own-skills-without-retraining-the
13. Demystifying agentic AI - Moody's, accessed April 9, 2026, https://www.moodys.com/web/en/us/creditview/blog/agentic-ai.html
14. The rise of agentic AI in financial services: from automation to autonomy - Moody's, accessed April 9, 2026, https://www.moodys.com/web/en/us/creditview/blog/agentic-ai-in-financial-services.html
15. I tried trading with Agentic AI, and it's mind blowing | by Itay V | InsiderFinance Wire, accessed April 9, 2026, https://wire.insiderfinance.io/i-tried-trading-with-agentic-ai-and-its-mind-blowing-455b88d3a56c
16. I just released my new open-source trading system using multi-agent AI approach - Reddit, accessed April 9, 2026, https://www.reddit.com/r/algotrading/comments/1nf6ghg/i_just_released_my_new_opensource_trading_system/

17. PageIndex: The RAG Framework That Threw Out Vector Databases and Still Hit 98.7% Accuracy - Towards AI, accessed April 9, 2026, https://pub.towardsai.net/pageindex-the-rag-framework-that-threw-out-vector-databases-and-still-hit-98-7-accuracy-d194e0549478
18. PageIndex Deep Dive: The Good, The Bad, and The Ugly of Vectorless RAG - sjramblings.io, accessed April 9, 2026, https://sjramblings.io/pageindex-deep-dive-vectorless-rag/
19. PageIndex vs Traditional RAG: A Better Way to Build Document Chatbots - Analytics Vidhya, accessed April 9, 2026, https://www.analyticsvidhya.com/blog/2026/03/pageindex-vs-rag-document-chatbots/
20. Vectorless RAG Hits 98.7% Accuracy: PageIndex Challenges Vectors | byteiota, accessed April 9, 2026, https://byteiota.com/vectorless-rag-pageindex-accuracy/
21. I tried Hermes so you don't have to. : r/openclaw - Reddit, accessed April 9, 2026, https://www.reddit.com/r/openclaw/comments/1se64gt/i_tried_hermes_so_you_dont_have_to/
22. Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions, accessed April 9, 2026, https://arxiv.org/html/2510.25445v1
23. PageIndex and RAG Comparative Study | by Shailesh Chaudhary | Mar, 2026 | Medium, accessed April 9, 2026, https://medium.com/@shailesh16221/pageindex-and-rag-comparative-study-815db06c92aa
24. Vectorless RAG: How PageIndex Works (2026 Guide) - Build Fast with AI, accessed April 9, 2026, https://www.buildfastwithai.com/blogs/vectorless-rag-pageindex-guide
25. PageIndex as RAG Alternative: A Comparative Evaluation of Vectorless Document Retrieval, accessed April 9, 2026, https://www.nateking.dev/blog/pageindex-rag-alternative
26. No, PageIndex Will Not “Kill” RAG, But It Is Indeed Excellent In Some Cases - Medium, accessed April 9, 2026, https://medium.com/@aldendorosario/no-pageindex-will-not-kill-rag-but-it-is-indeed-excellent-in-some-cases-11bc67473145
27. Introducing LightRAG: A New Era in Retrieval Augmented Generation | by Isham Rashik | Knowledge-Nexus-AI | Medium, accessed April 9, 2026, https://medium.com/knowledge-nexus-ai/introducing-lightrag-a-new-era-in-retrieval-augmented-generation-57c20d6081fa
28. The Hidden Cost of 98% Accuracy: A Practical Guide to RAG Architecture Selection - Tao An, accessed April 9, 2026, https://tao-hpu.medium.com/the-hidden-cost-of-98-accuracy-a-practical-guide-to-rag-architecture-selection-6883adc5289c
29. sec-api-python/README.md at master · janlukasschroeder/sec-api ..., accessed April 9, 2026, https://github.com/janlukasschroeder/sec-api-python/blob/master/README.md
30. SEC EDGAR Filings API, accessed April 9, 2026, https://sec-api.io/
31. GitHub - stefanoamorelli/sec-edgar-agentkit: AI agent toolkit for accessing and

analyzing SEC EDGAR filing data. Build intelligent agents with LangChain, MCP-use, Gradio, Dify, and smolagents to analyze financial statements, insider trading, and company filings., accessed April 9, 2026, https://github.com/stefanoamorelli/sec-edgar-agentkit
32. GitHub - wescules/insider-trading-analyzer: A Python project that downloads Form 4 and Form 144 SEC filings, filters for high-signal insider trades, cross-references market signals, and supports backtesting., accessed April 9, 2026, https://github.com/wescules/insider-trading-analyzer
33. Twitter (X) Tweets Scraper API - Apify, accessed April 9, 2026, https://apify.com/scraper-engine/twitter-x-tweets-scraper-scraper/api
34. Integrations | Hermes Agent - nous research, accessed April 9, 2026, https://hermes-agent.nousresearch.com/docs/integrations/
35. Hermes Agent: Self-Improving AI with Persistent Memory | YUV.AI Blog, accessed April 9, 2026, https://yuv.ai/blog/hermes-agent
36. What Is Agentic Architecture? | IBM, accessed April 9, 2026, https://www.ibm.com/think/topics/agentic-architecture
37. Building an Agentic AI Trading System from End to End (Updated 2026) | by Pranjal Saxena, accessed April 9, 2026, https://medium.com/predict/building-an-agentic-ai-trading-system-from-end-to-end-0fbc0a95b2e2
38. LLM Wiki vs RAG Knowledge Base: Karpathy Approach Explained - Atlan, accessed April 9, 2026, https://atlan.com/know/llm-wiki-vs-rag-knowledge-base/
39. Risks for AI in Finance and a Proposed Agent-based Framework for Governance - arXiv, accessed April 9, 2026, https://arxiv.org/html/2512.11933v1
40. Agentic AI for Finance: Workflows, Tips, and Case Studies, accessed April 9, 2026, https://rpc.cfainstitute.org/research/the-automation-ahead-content-series/agentic-ai-for-finance
41. Closed-Loop Development: How AI Agents Build Software While You Sleep - Medium, accessed April 9, 2026, https://medium.com/@alexzanfir/closed-loop-development-how-ai-agents-build-software-while-you-sleep-6df42cd05a85
42. A practical guide to building agents - OpenAI, accessed April 9, 2026, https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf
43. [2604.05119] Governance-Aware Agent Telemetry for Closed-Loop Enforcement in Multi-Agent AI Systems - arXiv, accessed April 9, 2026, https://arxiv.org/abs/2604.05119
44. Under the Covers With LightRAG: Extraction - Neo4j, accessed April 9, 2026, https://neo4j.com/blog/developer/under-the-covers-with-lightrag-extraction/
45. NousResearch/hermes-agent: The agent that grows with you - GitHub, accessed April 9, 2026, https://github.com/nousresearch/hermes-agent